

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:

Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Índice general

1. Introducción a Circom	3
1.1. Zero Knowledge Proof	3
1.2. R1CS	4
1.3. CIRCOM	5
2. Descripción del problema	7
2.1. Ejemplo del problema	7
2.2. Reglas de transformación	7
2.3. Ejemplo con constraints generadas automáticamente	7
3. Implementación	8
3.1. Explicación y ejemplo de la construcción de las reglas	8
4. Experimentos	9
4.1. Aplicación de la generación automática sobre juegos simples	9
4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints	9
4.3. Caso de uso, verificación y búsqueda de bugs	9

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que **una parte (prover) demuestre a otra (verifier)** la validez de una afirmación **sin revelar información sobre la misma**, mas allá de confirmar que es cierta.

Para ilustrar mejor este concepto, *se plantea la siguiente situación.*

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, **obtiene una prueba de que Alejandro es conocedor de esa información.**

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

No obstante, si observamos $0,1 * 0,1 = 0,01$, se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Con este ejemplo, el concepto de **ZKP** queda ilustrado de una forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográfico, es necesario formalizar estas interacciones mediante protocolos específicos, los cuales garantizan seguridad, robustez y eficiencia.

Históricamente, las ZKP crecieron desde su aparición como un simple concepto teórico en los años 80, hasta su desarrollo en la práctica. Actualmente, destacan dos grandes familias de protocolos:

- *Succinct Non-Interactive Argument of Knowledge*, mayormente conocido por sus siglas **zk-SNARKs**, que destacan por su **reducido tamaño de prueba**, **rapidez de verificación**, no se necesita un intercambio constante entre prover y verifier, además de garantizar la imposibilidad de que un prover genere una prueba falsa.
- *Scalable Transparent Argument of Knowledge*, conocido por sus siglas **zk-STARKs**, proporciona **transparencia** al no requerir una configuración de confianza inicial para generar claves, así como **escalabilidad** y **resistencia cuántica**.

Dada la complejidad que implica implementar estos protocolos desde cero, **se han desarrollado lenguajes de programación especializados**. *El objetivo es que desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar*. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. R1CS

Antes de hablar de Circom, **es importante conocer y comprender qué es R1CS**.

Rank-1 Constraints System, conocido como **R1CS**, es un sistema de representación matemática diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero.

En situaciones en las que múltiples partes deben validar de forma independiente una misma solución, R1CS permite generalizar este cálculo. Técnicamente, se acepta un coste adicional en la generación de la prueba con el fin de minimizar considerablemente el coste de la verificación. R1CS actúa como lenguaje que traduce la lógica de un programa a una estructura matemática (circuitos algebraicos) que un protocolo criptográfico puede procesar.

R1CS consigue esto gracias a que las restricciones deben seguir un formato de ecuación específico que no puede exceder el grado 2.

Esto implica que cada restricción debe poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A * B + C = 0$$

Donde A, B y C son combinaciones lineales de las señales del circuito.

Gracias a R1CS, es posible que el código escrito en **Circom** se transforme en un sistema de restricciones, consiguiendo así que la verificación sea sumamente eficiente.

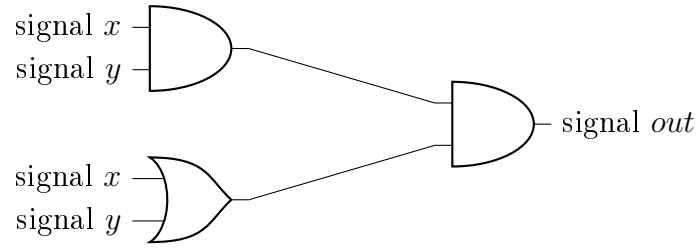
1.3. CIRCOM

Tras haber introducido los dos apartados anteriores introducidos, hasta este punto somos conocedores de que Circom es un lenguaje de programación basado en la descripción de circuitos, cuyo objetivo no es el cálculo como puede serlo en otro tipo de lenguajes, sino que **su propósito es conseguir demostrar sin revelar información sobre las operaciones realizadas**.

Como hemos visto anteriormente, Circom es la **herramienta que permite trasladar la teoría de las pruebas de conocimiento cero a un ámbito práctico**. Esto es posible **gracias a R1CS**, cuyo formato de las restricciones es necesario para que los protocolos de prueba realicen verificaciones eficientes.

Además de su finalidad criptográfica, es importante entender la diferencia entre un lenguaje basado en circuitos a uno de propósito general, siendo Circom basado en circuitos donde la estructura del mismo debe ser estática; esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que así aseguramos que el sistema de restricciones R1CS que se genera en base a este circuito, sea fijo.

Por otro lado, Circom introduce el concepto de *señales*, que actúan como los "cables" del circuito, los cuales transportan valores a través de las puertas lógicas del mismo. Mientras que las variables pueden usarse de manera auxiliar, **las señales son las que finalmente formarán parte de las ecuaciones que el verificador comprobará**.



Un programa Circom, al ser ejecutado, genera dos tipos de archivo: *un archivo R1CS* y *un archivo Witness (Téstito)*. **R1CS** es el archivo en el que se añaden todas las restricciones del programa Circom, mientras que en **Witness** contiene el valor de cada señal del circuito, además de los valores intermedios.

Como hemos mencionado anteriormente, **las señales** son las que forman parte de las ecuaciones a verificar, o dicho de otra manera, **serán las restricciones que se añadirán al archivo R1CS para ser verificadas**. *Aquí entra en juego la finalidad del proyecto, la cual detallaremos en el siguiente capítulo.*

Descripción del problema

- 2.1. Ejemplo del problema
- 2.2. Reglas de transformación
- 2.3. Ejemplo con constraints generadas automáticamente

Implementación

3.1. Explicación y ejemplo de la construcción de las reglas

Experimentos

- 4.1. Aplicación de la generación automática sobre juegos simples
- 4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints
- 4.3. Caso de uso, verificación y búsqueda de bugs